UNIVERSITÉ DE MONTPELLIER

Compte rendu Projet

HAI819I - Moteur de jeux

Clément Potier, Isaac Lluís, Ryan Rodrigues
22505270, 21600516, 22107209

Sommaire

1. Introduction 2
2. HUD 2
   2.1. Collectible 2
   2.2. Flèche indicactrice de forces 2
   2.3. Détection de proximité 3
3. Physique 3
4. PBR 4
5. Graphe de scène 4
6. ECS 4
7. LOD 5
8. PVS 5
9. Prototype 6

1. Introduction

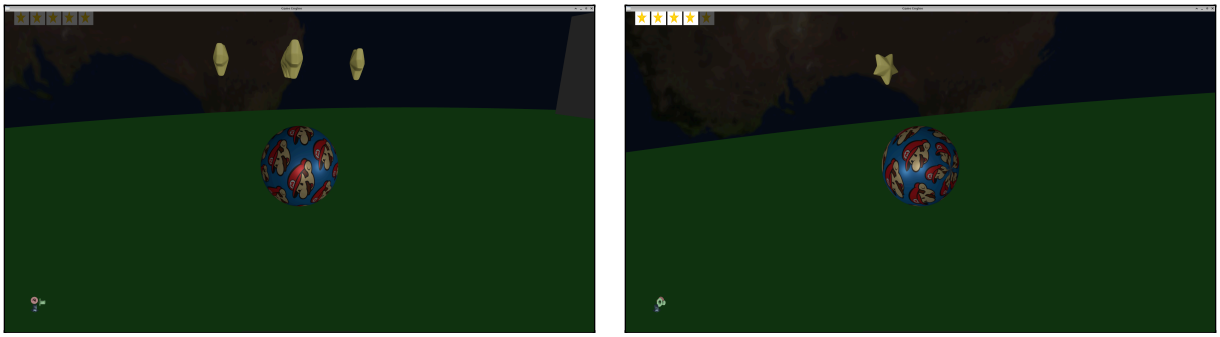
Ce dernier compte-rendu a pour but de passer en revue les différentes fonctionnalités implémentées dans ce moteur et ce, accompagnée d'images.

Le code pour ce projet est disponible sur ce répo du gitlab. Le code est sur la branche main.

2. HUD

2.1. Collectible

Le premier élément visible du HUD est un compteur de collectibles en haut à gauche, ici des étoiles pour rester sur le style Mario. Ce compteur est une texture importée sur l'espace écran et assombrie pour montrer qu'il n'a pas encore été collecté, mais qui est illuminée une fois l'objet est récolté. Quand le personnage s'approche d'une étoile collectible, l'étoile disparaît et le HUD se met à jour.



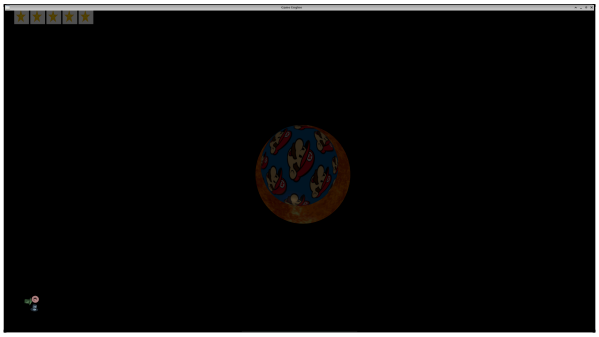
Avant collecte Après collecte

Fig. 1 — Démonstration du HUD des collectibles (haut gauche)

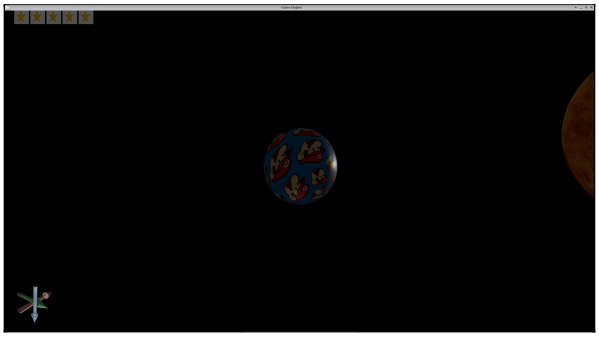
2.2. Flèche indicactrice de forces

Nous avons décidé de mettre un indicateur de la vitesse du personnage (des flèches) en bas à gauche de l'écran pour mieux visualiser les déplacements de celui-ci. Chaque flèche est

représenté par une couleur différente et a une taille proportionnelle à la vitesse dans cette direction.



Flèches à l'arrêt

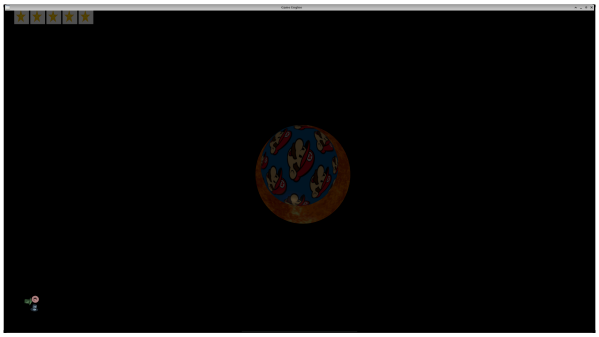


Flèches en mouvement

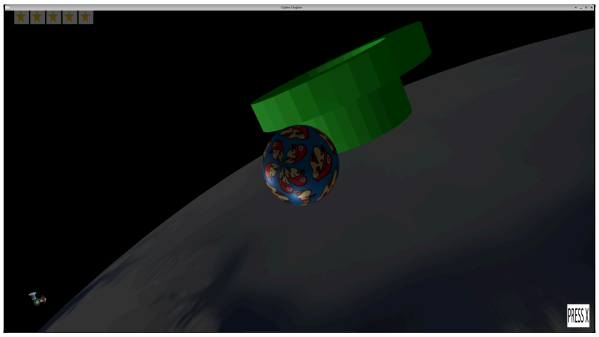
Fig. 2 — Démonstration du HUD de l'indicateur de vitesse (bas gauche)

2.3. Détection de proximité

Le dernier élément du HUD est un pop-up en bas à droite qui est déclenché quand le joueur est proche d'un objet définissable. Dans le prototype, celui-ci indique que l'utilisateur peut appuyer sur une touche pour entrer dans la planète.



Eloignée d'un tuyau

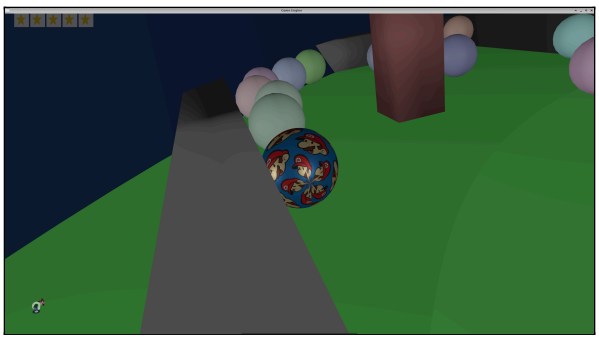


Proche d'un tuyau

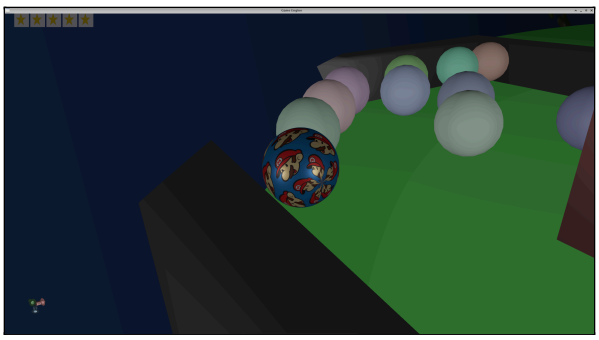
Fig. 3 — Démonstration du HUD de proximité (bas droite)

3. Physique

Nous avons implémenté un moteur physique basique en utilisant l'intégration de Verlet. Ce moteur physique permet d'avoir de la gravité, de la friction de l'air et tous les mouvements de base en appliquant de la force sur le personnage. Il permet de gérer la collision entre deux types d'objets, les sphères et les hexaèdre (objets avec six faces et huit sommets).



Avant collision

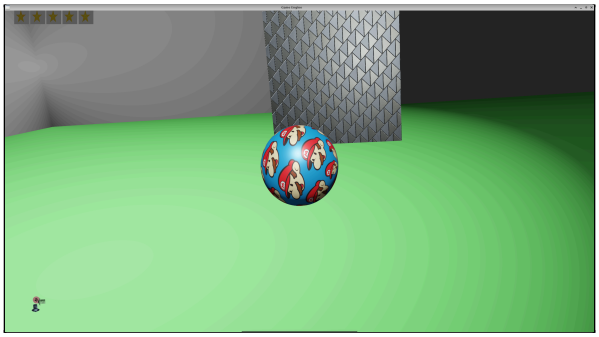


Après collision

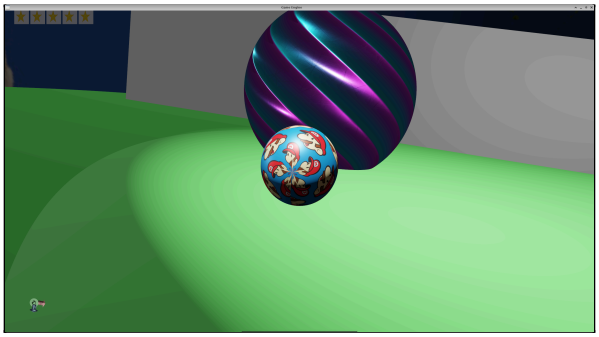
Fig. 4 — Démonstration des collisions.

4. PBR

On a implémenté le rendu physiquement réaliste permettant donner une texture d'albedo, métallicité, rugosité et occlusion ambiante à n'importe quel objet. Les shaders prennent correctement en compte les lumières présentes dans la scène.



Un cube futuristique



Une décoration de noël

Fig. 5 — Des objets avec un rendu physiquement réaliste

5. Graphe de scène

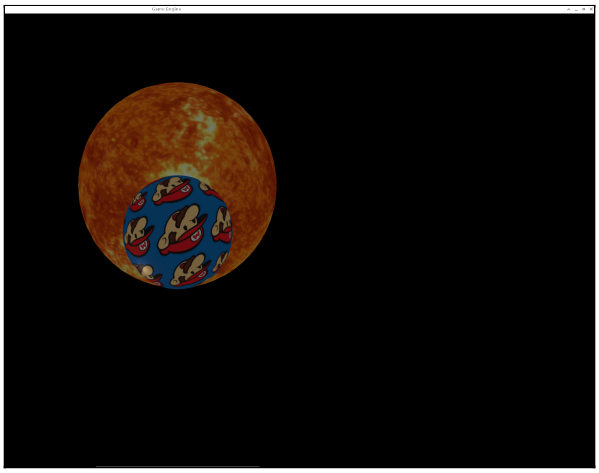
Afin de gérer les transformations, nous avons fait un graphe de scène. Celui-ci permet de gérer les scènes dans leur entièreté, c'est-à-dire qu'on peut choisir d'afficher ou non une instance d'un graphe de scène. Il permet d'ajouter et retirer des modèles, en créant une arborescence dans ceux-ci. Les transformations peuvent être appliqués dans l'arborescence pour qu'elles se propagent. Une attention particulière a été portée à la qualité du code de cette feature, pour avoir une implémentation flexible et robuste.

6. ECS

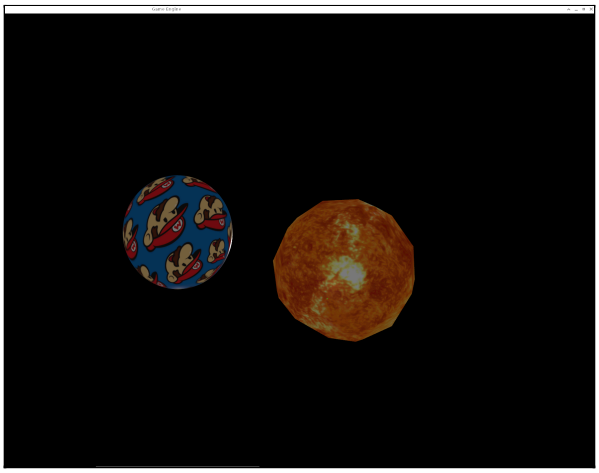
Le moteur a été programmé en utilisant le patron de conception ECS. Plein de systèmes l'utilisent, tel que le moteur physique, les collectibles, les LOD, etc. Il y a même un système dont le rôle est de mettre à jour les autres systèmes automatiquement, faisant que certains systèmes sont aussi des entités. Cette implémentation a été faite en cherchant à optimiser les performances et éviter les cache-miss.

7. LOD

Pour simplifier les meshs éloignés, on a implémenté un Continuous LOD (CLOD) avec pour métrique de simplification la taille que prend l'objet à l'écran. Plusieurs seuils définissent le niveau de dégradation du mesh, qui utilise une technique de partitionnement.



Soleil proche



Soleil après le premier seuil de LOD

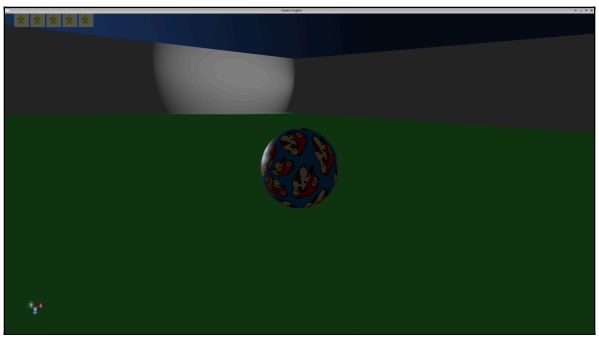


Soleil après le deuxième seuil de LOD

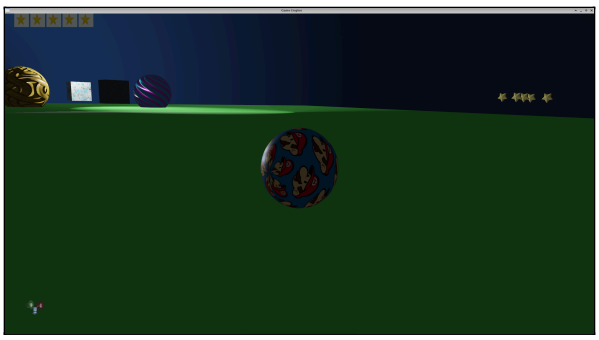
Fig. 6 — Vu des différentes versions du Mesh du soleil

8. PVS

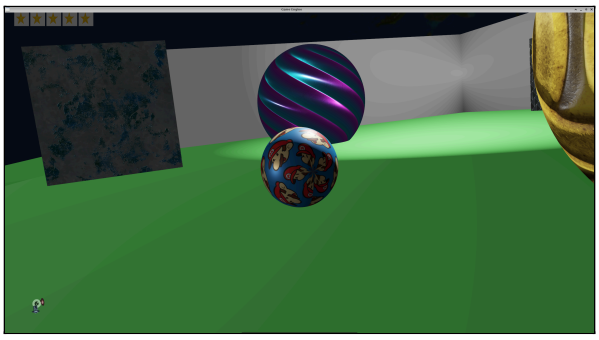
Le manager de PVS est en fait un graphe non-orienté, où chaque noeud est un noeud du graphe de scène. Ce système permet de gérer l'entrée et la sortie dans une salle (donc un noeud du PVS), affichant et cachant les éléments dans les noeuds adjacents.



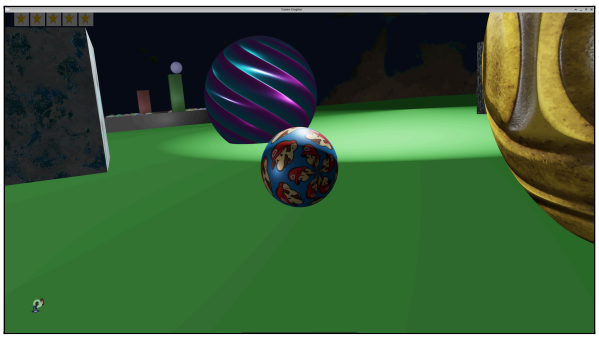
La salle de départ



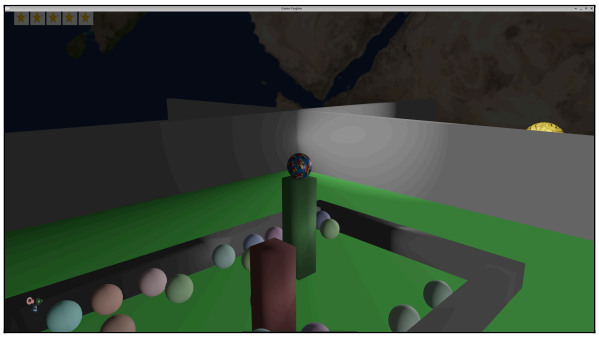
La salle de départ avec les murs invisibles



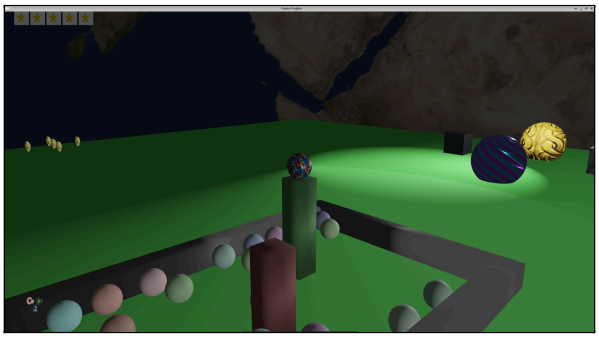
La salle PBR



La salle PBR avec les murs invisibles



La salle de piscine de boules



La salle de piscine de boules avec les murs invisibles

Fig. 7 — Dans la salle de départ, la piscine de boules n'est pas visible. Dans la salle PBR, la piscine de boules est visible, mais les étoiles deviennent invisibles. Dans la salle de la piscine de boules, les deux salles étant adjacentes, tous les modèles sont visibles.

9. Prototype

Nous avons fait un prototype simple permettant de présenter les différentes features décrites dans ce rapport. Voir la vidéo pour plus de détails: https://filesender.renater.fr/?s=download&token=70a499d6-7ebc-4c54-813f-fbd7f2008790